

# Outfit Lab

Dress one character together.

Every save, merge, send and undo runs real Git.



## Outfit Lab

**Open:** \_\_\_\_\_

Open the link and type your name.  
The app puts you in a lab.



QR

**STEP 1****Save every version**

# git commit

**WHAT IT IS**

A saved snapshot of the whole project.

**WHAT IT DOES**

Records the exact state, who saved it, when, and the commit before it.

**HOW GIT DOES IT**

Each file is stored as a blob, the folder as a tree. A commit holds the tree ID, parent ID(s), author, committer and message. Its ID is the SHA-1 of all that, so any change gives a new ID. Nothing is edited in place.

**Git records the name and clock your laptop gives it, and checks neither.**

**STEP 2****Give each idea its own branch**

# git branch

**WHAT IT IS**

A movable label on one commit.

**WHAT IT DOES**

Lets you work on an idea without touching main.

**HOW GIT DOES IT**

A branch is a tiny file (`.git/refs/heads/<name>`) holding one commit ID. Committing moves it forward. HEAD records which branch you are on. Creating a branch copies nothing.

**In real Git, each of you would have your own clone with one working directory. Here your lab shares one repository.**

**STEP 3****Combine two branches**

# git merge

**WHAT IT IS**

Combining two lines of work into one.

**WHAT IT DOES**

Keeps a change made on one side. It stops for a person when both sides changed the same or neighbouring lines.

**HOW GIT DOES IT**

Git finds the merge base (the last commit both sides share), compares each side with it (a 3-way merge), and writes a merge commit with two parents. If your branch has nothing new, Git fast-forwards: only the label moves, and no merge commit is made.

**STEP 4****Share your work through the Wall**

# git push / git pull

**WHAT IT IS**

Sharing commits with a copy elsewhere, here the Wall.

**WHAT IT DOES**

push sends your new commits and moves the remote branch.  
pull brings the remote's new commits into yours.

**HOW GIT DOES IT**

Copies exchange only the objects the other side lacks; IDs come from content, so copies agree without coordination. A push is accepted only if it is a fast-forward of the remote branch. Otherwise you fetch first, then merge or rebase.

**STEP 4****Share your work through the Wall**

# git rebase

**WHAT IT IS**

Replaying your commits on top of another branch.

**WHAT IT DOES**

Gives a straight history with no merge commit.

**HOW GIT DOES IT**

For each of your commits, Git applies its change to the new base and writes a new commit. The new parent gives it a new ID. The author and author date are kept; the committer date is new. The originals become unreachable and stay in your reflog for a while.

**STEP 5****Undo a change everyone already has**

# git revert / git reset

**WHAT IT IS**

Two kinds of undo.

**WHAT IT DOES**

revert adds a commit that undoes an old one, and is safe on shared history. reset moves your branch label back, and is safe only if nobody else has the commits you leave behind.

**HOW GIT DOES IT**

revert applies the opposite of the commit's change and commits it on top, so history only grows. reset rewrites the branch file. The commits left behind are still listed in the reflog (a local log of where each label pointed) until they expire and gc removes them.

**STEP 6****Rewrite the shared history**

# Squash + `git push --force`

**WHAT IT IS**

Cleaning up history by rewriting it.

**WHAT IT DOES**

Squash turns several commits into one. A force push makes the Wall accept the rewritten branch.

**HOW GIT DOES IT**

Squash writes one new commit whose tree is the final state. The originals, with their authors and times, become unreachable. `--force` skips the fast-forward check and moves the remote branch. `git gc` later deletes unreachable objects.

## Switching to Git: the Good, the Bad, and the Ugly

### GOOD

Branches are cheap.

Commits and reverts are local.

Developers prefer a flat history.

### BAD

Fast-forward forgets which branch a change came from.

Rebase rewrites the commit; the patch can change too.

Squash drops the individual commits. Once the branch is deleted, it drops who made them and when.

### UGLY

Microsoft traces each change's route to main: its integration path.

For Git, that tracing had to be redesigned from scratch.

Some of the loss cannot be recovered.

## For analysts, flat history is data loss.

Flat history helps developers find and revert a bad change (bisect, revert). It loses where a change came from and who made it.

## THE PAPER

### What you lived

Step 3: after the fast-forward, you deleted the fancy branch. No card records which cards were made on it.

Step 4: Replay on top made a copy with a new ID. The original card is not on the Wall.

Step 6: the squash dropped the individual cards, and with them who made each one.

Step 6: the 🥾 audit failed. The Wall no longer knows who added the boots.

PAIRS, IN THE APP

**Your team squash-merges every feature branch and deletes it. What do you gain, and what can an auditor no longer answer?**

**A commit never changes.**

**A branch is a pointer that moves.**

**The Wall holds copies of commits.**